

assignmentA

DSA Assignment #A : 5月份月考

Updated 2026-05-12 11:16 GMT+8

Compiled by 徐前 物理学院 (2026 Spring)

说明:

1. 解题与记录:

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge, Codeforces, LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排:** 提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交:** 如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

E04137: 最小新整数

monotonic stack, <http://cs101.openjudge.cn/practice/04137/>

思路:

如果一个数比前面的数小，就能把它替代，并且替代后的总体更小，用单调栈实现。

代码:

```
def main():
    t = int(input())
    for _ in range(t):
        n, k = input().split()
        k = int(k)
        st = []
        for i, c in enumerate(n):
```

```

        while st and k and st[-1] > c:
            st.pop()
            k -= 1
        if k == 0:
            st.append(n[i:])
            break
        st.append(c)
        for _ in range(k):
            st.pop()
        print(''.join(st))
    return

if __name__ == '__main__':
    main()

```

代码运行截图 (至少包含有"Accepted")

#52496026提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```

def main():
    t = int(input())
    for _ in range(t):
        n, k = input().split()
        k = int(k)
        st = []
        for i, c in enumerate(n):
            while st and k and st[-1] > c:
                st.pop()
                k -= 1
            if k == 0:
                st.append(n[i:])
                break
            st.append(c)
        for _ in range(k):
            st.pop()
        print(''.join(st))

```

基本信息

#: 52496026
 题目: 04137
 提交人: 25n2500011619(xq)
 内存: 3556kB
 时间: 21ms
 语言: Python3
 提交时间: 2026-05-08 21:11:39

E04143: 和为给定数

two pointers, <http://cs101.openjudge.cn/dsapre/04143/>

思路:

对于某一个数 x , 哈希表中存它的配对数 $m - x$, 找最小的一对。

代码:

```

def main():
    n = int(input())
    a = list(map(int, input().split()))
    m = int(input())

```

```

a.sort()
st = set()
ans = -1
for x in a:
    if m - x in st:
        ans = m - x
    st.add(x)

if ans == -1:
    print('No')
else:
    print(ans, m - ans)

if __name__ == '__main__':
    main()

```

代码运行截图（至少包含有"Accepted"）

#52496045提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```

def main():
    n = int(input())
    a = list(map(int, input().split()))
    m = int(input())
    a.sort()
    st = set()
    ans = -1
    for x in a:
        if m - x in st:
            ans = m - x
        st.add(x)

    if ans == -1:
        print('No')
    else:

```

基本信息

#: 52496045
 题目: 04143
 提交人: 25n2500011619(xq)
 内存: 16744kB
 时间: 67ms
 语言: Python3
 提交时间: 2026-05-08 21:13:29

M27638: 求二叉树的高度和叶子数目

<http://cs101.openjudge.cn/practice/27638/>

思路:

按照题意建树，然后找根节点，递归求高度和叶子数。

代码:

```

class Node:
    def __init__(self, value):
        self.value = value

```

```

        self.left = None
        self.right = None
        self.height = 0
        self.count = 0

def dfs(node):
    if node.left is None and node.right is None:
        node.count += 1
        return

    if node.left:
        dfs(node.left)
        node.count += node.left.count
        node.height = node.left.height + 1
    if node.right:
        dfs(node.right)
        node.count += node.right.count
        node.height = max(node.height, node.right.height + 1)
    return

def main():
    n = int(input())
    nodes = [Node(i) for i in range(n)]
    ind = [0] * n
    for i in range(n):
        left, right = map(int, input().split())
        if left != -1:
            nodes[i].left = nodes[left]
            ind[left] += 1
        if right != -1:
            nodes[i].right = nodes[right]
            ind[right] += 1
    root = ind.index(0)
    dfs(nodes[root])
    print(nodes[root].height, nodes[root].count)

if __name__ == '__main__':
    main()

```

代码运行截图 (至少包含有"Accepted")

#52496069提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
        self.height = 0
        self.count = 0

    def dfs(node):
        if node.left is None and node.right is None:
            node.count += 1
            return

        if node.left:
            dfs(node.left)
            node.count += node.left.count
            node.height = node.left.height + 1
```

基本信息

#: 52496069
题目: 27638
提交人: 25n2500011619(xq)
内存: 3688kB
时间: 20ms
语言: Python3
提交时间: 2026-05-08 21:16:26

M30720: 败方树的构建与维护

<http://cs101.openjudge.cn/practice/30720/>

思路:

建一棵完全二叉树，每个节点存储比赛的两个值，小的在前；父节点从子节点的两个小值继承。更新时从底部向上更新，时间复杂度 $O(\log n)$ 。

代码:

```
def output(heap, n):
    res = [heap[1][0]]
    for i in range(1, n):
        res.append(heap[i][1])
    print(*res, sep=' ')

def main():
    n, m = map(int, input().split())
    arr = list(map(int, input().split()))
    heap = [[] for _ in range(n << 1)]
    # flag = [[] for _ in range(n)]
    opt = (1 << n.bit_length()) - n
    for i in range(n, n << 1):
        heap[i].append(arr[(i - opt) % n])
    for i in range(n - 1, 0, -1):
        heap[i] = [heap[i << 1][0], heap[i << 1 | 1][0]]
        if heap[i][0] > heap[i][1]:
            heap[i].reverse()
    output(heap, n)
```

```

for _ in range(m):
    idx, val = map(int, input().split())
    i = (idx + opt) % n + n
    heap[i][0] = val
    i = i >> 1
    while i >= 1:
        heap[i] = [heap[i << 1][0], heap[i << 1 | 1][0]]
        if heap[i][0] > heap[i][1]:
            heap[i].reverse()
        i >>= 1
    output(heap, n)

if __name__ == '__main__':
    main()

```

代码运行截图 (至少包含有"Accepted")

#52496161提交状态

查看 提交 统计 提问

状态: Accepted

源代码

```

def output(heap, n):
    res = [heap[1][0]]
    for i in range(1, n):
        res.append(heap[i][1])
    print(*res, sep=' ')

def main():
    n, m = map(int, input().split())
    arr = list(map(int, input().split()))
    heap = [[] for _ in range(n << 1)]
    # flag = [[] for _ in range(n)]
    opt = (1 << n.bit_length()) - n
    for i in range(n, n << 1):
        heap[i].append(arr[(i - opt) % n])
    for i in range(n - 1, 0, -1):
        heap[i] = [heap[i << 1][0], heap[i << 1 | 1][0]]

```

基本信息

#: 52496161
 题目: 30720
 提交人: 25n2500011619(xq)
 内存: 6872kB
 时间: 200ms
 语言: Python3
 提交时间: 2026-05-08 21:28:21

27093: 排队又来了

Segment Tree, Discretization (离散化), binary search,

<http://cs101.openjudge.cn/practice/27093/>

思路:

对于任意两个不能交换 (绝对值大于 d) 的数, 可以建一条有向边, 然后进行拓扑排序, 每一次选出入度为 0 的一组数, 它们之间可以任意交换, 那么字典序最小的方案就是将它们从小到大排列, 然后将它们从图中删除, 进行下一轮选择。

由于每个数可能连多条边, 图可能非常稠密, 手动建边不可行, 考虑类似词梯中分组的思想: 对每个数分层, 层内的数可以任意交换, $Layer[i]$ 中的每个数与 $Layer[i - 1]$ 至少连一条边。那么本题的关键就是计算每个数的 $Layer$ 。

于是问题转化为计算一段区间 $(x + d, +\infty)$ 与 $(-\infty, x - d)$ 的 *Layer* 最大值，由于数据大小范围 (10^9) 远大于数的个数 N 范围，需要先进行离散化，单点修改+区间查找最大值 → 用树状数组 或 线段树实现。

代码

```
import sys
import bisect
from collections import defaultdict

class BIT:
    def __init__(self, n):
        self.n = n
        self.tree = [0] * (n + 1)
    def add(self, i, x) -> None:
        while i <= self.n:
            self.tree[i] = max(x, self.tree[i])
            i += i & -i
    def pre_sum(self, i) -> int:
        res = 0
        while i > 0:
            res = max(res, self.tree[i])
            i &= (i - 1)
        return res

def main():
    it = iter(sys.stdin.read().split())
    n, d = int(next(it)), int(next(it))
    h = [int(next(it)) for _ in range(n)]
    vals = sorted(set(h))
    m = len(vals)
    rank = {x : i for i, x in enumerate(vals)}
    bitl = BIT(m)
    bitr = BIT(m)
    max_layer = 0
    layer_buckets = defaultdict(list)
    for x in h:
        l = bisect.bisect_left(vals, x - d)
        left_max = bitl.pre_sum(l) if l else 0
        r = bisect.bisect_right(vals, x + d)
        right_max = bitr.pre_sum(m - r) if r < m else 0

        cur_layer = max(left_max, right_max) + 1
        max_layer = max(max_layer, cur_layer)
        layer_buckets[cur_layer].append(x)
```

```

        bitl.add(rank[x] + 1, cur_layer)
        bitr.add(m - rank[x], cur_layer)
    ans = []
    for layer in range(1, max_layer + 1):
        ans.extend(sorted(layer_buckets[layer]))
    # print(*ans, sep = ' ')
    sys.stdout.write(' '.join(map(str, ans)))

if __name__ == '__main__':
    main()

```

(至少包含有"Accepted")

#52496466提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```

import sys
import bisect
from collections import defaultdict

class BIT:
    def __init__(self, n):
        self.n = n
        self.tree = [0] * (n + 1)
    def add(self, i, x) -> None:
        while i <= self.n:
            self.tree[i] = max(x, self.tree[i])
            i += i & -i
    def pre_sum(self, i) -> int:
        res = 0
        while i > 0:
            res = max(res, self.tree[i])
            i &= (i - 1)

```

基本信息

#: 52496466
 题目: 27093
 提交人: 25n2500011619(xq)
 内存: 61808kB
 时间: 532ms
 语言: PyPy3
 提交时间: 2026-05-08 22:25:21

T30669: 地铁换乘

LCA, binary lifting, <http://cs101.openjudge.cn/practice/30669/>

思路:

这道题因为只有单组查询，所以可以直接搜索得到 p 到 q 的路径，记录长度并处以两人的相对速度得到时间，然后找到路径中对应的会合点即可。

代码

```

def main():
    def dfs(u, pre):
        depth[u] = depth[pre] + 1
        for v in edge[u]:
            if v != pre:
                dfs(v, u)

```



```

n, t = map(int, input().split())
edge = [[] for _ in range(n + 1)]
depth = [0] * (n + 1)
for _ in range(n - 1):
    a, b = map(int, input().split())
    edge[a].append(b)
    edge[b].append(a)
p, q, v1, v2 = map(int, input().split())
depth[0] = -1
dfs(t, 0)
st = [p]
dis = [-1] * (n + 1)
prev = [-1] * (n + 1)
dis[p] = 0
prev[p] = 0
while st:
    u = st.pop()
    if u == q:
        break
    for v in edge[u]:
        if v != prev[u]:
            dis[v] = dis[u] + 1
            prev[v] = u
            st.append(v)
l = dis[q]
day = l // (v1 + v2)
s = q
for i in range(day * v2):
    s = prev[s]
print(day, depth[s])

if __name__ == '__main__':
    main()

```

(至少包含有"Accepted")

#52496217提交状态

查看 提交 统计 提问

状态: Accepted

源代码

```
def main():
    def dfs(u, pre):
        depth[u] = depth[pre] + 1
        for v in edge[u]:
            if v != pre:
                dfs(v, u)

    n, t = map(int, input().split())
    edge = [[] for _ in range(n + 1)]
    depth = [0] * (n + 1)
    for _ in range(n - 1):
        a, b = map(int, input().split())
        edge[a].append(b)
        edge[b].append(a)
    p, q, v1, v2 = map(int, input().split())
    depth[0] = -1
    dfs(t, 0)
```

基本信息

#: 52496217
题目: 30669
提交人: 25n2500011619(xq)
内存: 42076kB
时间: 740ms
语言: Python3
提交时间: 2026-05-08 21:37:18

2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2026spring每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

本次月考 AC4，第4题在“轮空”情形卡住了，耗了很久后放弃去看第6题，发现比较简单。第5题上学期做过，但是没仔细研究过 $O(n \log n)$ 的优化算法，考场上没写完。

回去以后重新写了一遍排队加强版，发现对于离散化和树状数组/线段树求区间最值这类问题还是不熟悉，特别是离散化过程中对于 `sorted`, `set`, `dict`, `defaultdict`, `bisect_left/right` 这些函数和数据类型的熟练应用，自己过一遍整个代码对于理解有很多帮助。

排队这道题有意思的一点在于它和词梯一样都是需要两两比较才能建边的，所以建边时间复杂度很大，处理的思路也有点相似：把相同“性质”的点都放进一个 *Bucket* 或 *Layer* 中，然后把这个桶或层作为一个整体去处理我们要的结果。这也有点像强连通单元的思想。